



Unifor

T1

Grupo D: Problema dos Dominós

Alunos: Gabriel Rocha (2410451), Nicolas Marcelino (2420508)

Orientador: Prof. Me. Ricardo Carubbi

Disciplina: Resolução de Problemas com Grafos

Centro de Ciências Tecnológicas - CCT

Universidade de Fortaleza - UNIFOR



Introdução

O cenário:

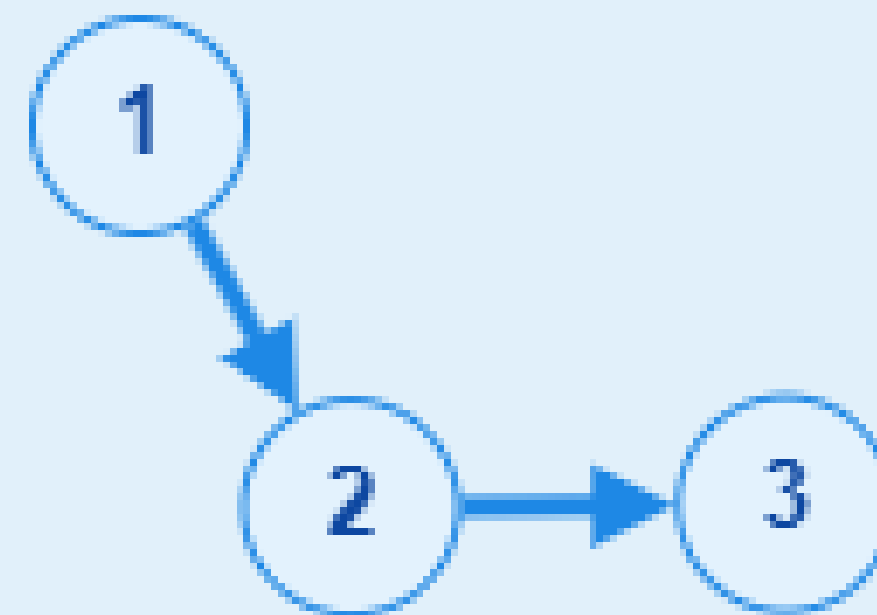
- O problema apresenta uma sequência de dominós organizados em que alguns são derrubados manualmente.
- A queda de um dominó pode se espalhar para outros conforme as relações estabelecidas.
- Cada relação define que, se o dominó A cair, o dominó B também cairá



Modelagem

Grafo direcionado, pois:

- o par ordenado (x, y) indica que a queda do dominó x provoca a queda do dominó y ;
- a direção das relações permite representar corretamente a propagação da queda entre os dominós.



Grafo baseado no caso de teste disponibilizado pela plataforma. Fonte: elaborado pelos autores com auxílio do Graph Online.



Representação computacional

Lista de adjacência, pois:

- aloca apenas as relações existentes;
- responde à tendência do grafo ser esparso;

Adjacency List	
0:	1
1:	2
2:	

Lista de adjacência baseada no caso de teste disponibilizado pela plataforma. Fonte: elaborado pelos autores com auxílio do VisuAlgo.





Complexidade

A solução é ótima para o problema, pois processa a entrada em tempo linear em relação ao tamanho do grafo.

Aspecto	Complexidade	Descrição
Tempo DFS	$O(V + E)$	Cada vértice e aresta são visitados no máximo uma vez.
Espaço Grafo	$O(V + E)$	Lista de adjacência armazena todos os vértices e conexões.
Espaço DFS	$O(V)$	Pior caso da pilha de recursão e vetor de marcação.

V: Número total de dominós (vértices).

E: Número total de relações de enfileiramento (arestas).





Explicação da DFS no problema

Conceito DFS	Aplicação no Problema dos Dominós
Múltiplas Fontes	A busca inicia a partir de cada dominó derrubado manualmente (vetor de origens).
Recursão / Pilha	Simula a reação em cadeia : se o dominó A cai e empurra B, a DFS explora B imediatamente.
Vetor de Marcação	Garante que cada dominó seja contado apenas uma vez , mesmo se atingido por múltiplos caminhos.
Alcançabilidade	O resultado final (count) é o total de dominós que pertencem ao fecho transitivo das fontes.





Implementação (em Python)

```
while testes:

    derrubados = 0
    empurrados = []
    total_dominos, total_enfileirados, total_empurrados = input().split()
    total_dominos, total_enfileirados, total_empurrados = int(total_dominos), int(total_enfileirados), int(total_empurrados)

    g = Digraph(total_dominos) ## mapeando dominós em um grafo

    while total_enfileirados:

        x, y = input().split()
        x, y = int(x), int(y)

        g.add_edge(x - 1, y - 1) ## mapeando sequência de enfileiramento

        total_enfileirados = total_enfileirados - 1

    while total_empurrados:
        derrubado_manualmente = int(input())

        empurrados.append(derrubado_manualmente - 1)

        total_empurrados = total_empurrados - 1

    dfs = DirectedDFS(g, empurrados)
    derrubados += dfs.count
    print(derrubados)

    testes = testes - 1
```

- Fluxo principal da solução do problema. Ele lê cada caso de teste, constrói o grafo dos dominós, identifica quais foram derrubados manualmente e inicia a DFS em cada origem para descobrir quantos dominós caem pela propagação.





Implementação (em Python)

```
from digraph import Digraph

class DirectedDFS:

    def __init__(self, G, sources):
        self._marked = [False for _ in range(G.V)]
        self.count = 0
        for s in sources:
            s = int(s)
            if not self._marked[s]:
                self.dfs(G, s)

    def dfs(self, G, v):
        self._marked[v] = True
        self.count += 1
        for w in G.adj[v]:
            if not self._marked[w]:
                self.dfs(G, w)
```

- DirectedDFS fornece o vetor de marcação e o método de busca em profundidade. Métodos de leitura de arquivos foram removidos, pois a leitura das entradas é feita no código-fonte principal que importa a estrutura.

```
1 class Digraph:
2
3     def __init__(self, v=0):
4         self.V = v
5         self.E = 0
6         self.adj = [[] for _ in range(self.V)]
7
8     def add_edge(self, v, w):
9         v, w = int(v), int(w)
10        self.adj[v].append(w)
11        self.E += 1
```

- Digraph representa o grafo direcionado. Métodos como *degree()*, *max_degree()* e *number_of_self_loops()* foram removidos por não servirem à solução. A lista de adjacência também deixou de ser uma lista de *bags* (sacos) para ser uma “lista de listas”.





Validação

```
main.py x casos-de-teste.txt x
src > main.py > ...
1 from digraph import Digraph
2 from directed_dfs import DirectedDFS
3
4 testes = int(input())
5
6 while testes:
7     derrubados = 0
8     empurrados = []
9     total_dominos, total_enfileirados, total_empurrados =
10     total_dominos, total_enfileirados, total_empurrados
11
12     g = Digraph(total_dominos)
13
14     while total_enfileirados:
15
```

dados > casos-de-teste.txt

1	1
2	3 2 1
3	1 2
4	2 3
5	2

Problems Output Debug Console Terminal Ports

```
User@DESKTOP-HM0H0RR MINGW64 ~/RPG/T1 (main)
$ python src/main.py < dados/casos-de-teste.txt
2
```

Entrada

1
3 2 1
1 2
2 3
2

Saída esperada

2





Validação

Submission 20400428

Edited from 20400425

Edit and resubmit

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
02:53:17	Dominoes 2	✓ Accepted	0.18 s	Python 3	4/4
✓✓✓✓					

A solução foi aprovada em todos os casos de teste da plataforma.





Unifor

